



CODIPLAN

Piloter Claude Code pour construire le projet

Méthode d'autonomie, dispositif de vérification et backlog exécutable
des lots 0 à 3

Projet	CODIPLAN v1.2 — 173 j/h avec le volet éditeur
Accompagne	Le cahier des charges et le fichier CLAUDE.md
Date	18 août 2026

1. Ce qui est atteignable, et ce qui ne l'est pas

Autant le dire d'emblée : « **en toute autonomie et parfaitement** » n'est pas atteignable en un seul geste. Aucun agent, aujourd'hui, ne prend un cahier des charges de 46 pages et n'en sort une plateforme multi-société, hors-ligne, avec facturation d'abonnements, sans intervention humaine et sans défaut.

Ce n'est pas une limite de puissance, c'est une limite de nature. Trois raisons.

Le cahier des charges est incomplet par construction. Il décrit ce que le système doit faire, pas chacune des milliers de micro-décisions qu'implique sa réalisation. Un agent qui ne demande jamais rien invente — et il invente plausiblement, ce qui est pire qu'une erreur visible.

La dérive est cumulative. Sur une session longue, chaque approximation devient le socle de la suivante. Au bout de quelques milliers de lignes, l'écart avec l'intention initiale n'est plus rattrapable par une relecture : il faut réécrire.

« **Ça compile et les tests passent** » ne veut pas dire « **c'est juste** ». Surtout quand c'est le même agent qui a écrit le code et les tests.

Ce qui **est** atteignable, et qui vaut largement le détour : un développement où Claude Code écrit plus de 90 % du code, travaille en continu par blocs de plusieurs heures, et où votre intervention se limite à des points de contrôle espacés et à des arbitrages métier. Ce document décrit comment y arriver.

2. Le principe : l'autonomie vient de la vérifiabilité

C'est l'idée centrale, et tout le reste en découle.

Un agent est autonome exactement dans la mesure où il peut vérifier son propre travail sans vous.

Tant que la seule façon de savoir si le code est correct consiste à ce que vous le lisiez, l'agent doit s'arrêter à chaque étape. Dès qu'une commande répond « oui, c'est conforme » ou « non, voilà ce qui casse », il peut enchaîner seul pendant des heures — parce qu'il a un juge.

La conséquence pratique est contre-intuitive : **l'effort le plus rentable n'est pas d'écrire un prompt plus long, c'est de construire la boucle de vérification.** Une journée passée à mettre en place les tests, le jeu de données et la commande de vérification vous rend dix journées d'autonomie derrière.

Corollaire : n'écrivez jamais un critère d'acceptation que la machine ne peut pas trancher. « L'interface doit être ergonomique » n'est pas vérifiable. « Créer une machine depuis le mobile requiert 3 champs et pas un de plus » l'est.

3. Les six pièces du dispositif

Pièce 1 — Le dépôt préparé avant la première ligne de code métier

Ne demandez pas à Claude Code de commencer par une fonctionnalité. Faites-lui d'abord construire le terrain sur lequel il va se juger lui-même :

- projet Next.js + TypeScript strict + Tailwind + shadcn/ui, qui compile ;
- Prisma branché sur une base PostgreSQL locale, avec une première migration ;
- Vitest et Playwright configurés, avec un test bidon qui passe ;
- un script `pnpm verify` enchaînant typecheck, lint, tests, tests d'isolation et build ;
- un `prisma/seed.ts` produisant **deux sociétés** avec des données réalistes ;
- l'intégration continue qui lance `pnpm verify` à chaque commit.

Ce lot ne produit aucune valeur visible. C'est pourtant celui qui détermine tout le reste.

Pièce 2 — Le CLAUDE.md

Le fichier `CLAUDE.md` fourni avec ce guide se place à la racine du dépôt. Claude Code le lit à chaque session, sans qu'on le lui demande. Il contient ce qui ne doit jamais être renégocié : la stack, les neuf invariants, les commandes, la définition de « terminé », et — c'est le plus important — **la liste des sujets sur lesquels il doit s'arrêter et vous demander**.

Cette dernière section est le mécanisme de sécurité principal. Un agent qui sait quand ne pas décider est infiniment plus utile qu'un agent qui décide toujours.

Gardez ce fichier court. Un `CLAUDE.md` de 600 lignes est lu en diagonale, comme le seraient des conditions générales.

Pièce 3 — Le backlog découpé en tickets vérifiables

C'est le travail le plus rentable que vous ferez sur ce projet, et il est déjà largement fait : le cahier des charges numérote les règles de gestion (RG-INT-01, RG-SOC-02...), ce qui donne une correspondance directe entre une règle et un test.

Un bon ticket tient en cinq lignes et comporte toujours :

- 1. **le but**, en une phrase ;
- 2. **les références** au cahier des charges — chapitre, module, règles concernées ;
- 3. **les critères d'acceptation**, formulés de façon vérifiable par une machine ;
- 4. **les limites** — ce qui n'est explicitement pas dans le ticket ;
- 5. **la commande de vérification** attendue.

Le chapitre 6 de ce guide fournit les tickets des lots 0 à 3, soit le chemin critique jusqu'à la mise en service terrain.

Pièce 4 — Les quatre gardiens

Quatre familles de tests portent l'essentiel du risque de ce projet. Elles doivent exister **avant** le code qu'elles surveillent.

Gardien	Ce qu'il protège	Pourquoi lui
<code>tests/isolation/</code>	Un utilisateur de la société A n'atteint jamais une donnée de la société B, y compris en manipulant les identifiants dans les URL et les appels d'API	C'est le défaut le plus grave possible, et le seul qui soit à la fois invisible en développement et rédhibitoire à la vente
<code>tests/unit/money/</code>	XPF sans décimale, EUR à deux, aucune conversion ligne à ligne	Une erreur d'arrondi ne plante pas : elle produit des chiffres faux que personne ne remarque avant six mois
<code>tests/unit/calendar/</code>	Ducos ouvre le samedi, Koné non, les fériés sont paramétrés par site et parfois travaillés	Un calendrier codé en dur est l'erreur la plus naturelle à commettre, et elle fausse tous les indicateurs
<code>tests/e2e/offline/</code>	Une intervention complète en mode avion, puis synchronisation intégrale sans perte ni doublon	C'est la fonctionnalité la plus difficile du projet et celle qui casse le plus silencieusement

Ces quatre répertoires sont sanctuarisés : le `CLAUDE.md` interdit de les alléger pour faire passer une vérification.

Pièce 5 — Les points de contrôle humains

Un par lot, pas un par tâche. À chaque fin de lot, vous faites trois choses, en une heure :

- vous **utilisez** le produit — pas seulement lire le rapport, cliquer réellement ;
- vous vérifiez **un scénario métier de bout en bout** parmi ceux du chapitre 21 du cahier des charges ;
- vous lisez le **journal des décisions** (`docs/decisions/`) pour repérer un choix qui vous surprend.

Ce dernier point est le plus précieux. Une décision technique qui vous étonne à la lecture révèle presque toujours un malentendu métier — et il coûte cent fois moins cher de le lever à la fin du lot 2 qu'à la recette finale.

Pièce 6 — La mémoire du projet

Trois mécanismes, complémentaires :

- **CLAUDE.md** — les règles permanentes.
- **docs/decisions/** — pourquoi telle option a été retenue. Sans cela, Claude Code re-délibère à chaque session et peut prendre le choix inverse trois semaines plus tard.
- **La section 9 du CLAUDE.md, « erreurs à ne pas refaire »** — une ligne par erreur corrigée. C'est la mécanique d'apprentissage la plus simple et la plus efficace du dispositif : elle transforme une correction ponctuelle en règle permanente.

4. Le déroulé concret

4.1 Mise en place, une fois

```
C:\Claude\CODIPLAN\  
├─ CLAUDE.md           ← fourni avec ce guide  
├─ docs/  
│   ├── cahier-des-charges.md ← le .md déjà livré  
│   ├── backlog.md         ← les tickets du chapitre 6  
│   └─ decisions/  
└─ (le code viendra ici)
```

Puis, dans le dossier : **claude** — et la session démarre en ayant déjà tout le contexte.

4.2 Le rythme d'une session

Une session productive suit toujours la même forme.

Ouverture — vous annoncez le ticket, rien de plus :

Ticket L1-04. Lis d'abord **docs/backlog.md** et le §7/M1 du cahier des charges. Écris les tests avant le code. Termine par **pnpm verify**.

Déroulé — Claude Code travaille seul. Vous n'intervenez que s'il pose une question. Résistez à la tentation de commenter le code au fil de l'eau : cela le fait changer de direction en permanence et coûte plus que cela ne rapporte.

Clôture — vous demandez systématiquement :

Résume ce qui a été fait, ce qui reste ouvert, et les décisions que tu as prises sans me demander.

Cette dernière question est celle qui rattrape le plus de dérives. Un agent qui liste ses décisions implicites vous montre exactement où il a comblé un silence du cahier des charges.

4.3 Ce qui fait échouer une session

Erreur	Ce qui se passe	À la place
Demander un lot entier en un prompt	Il commence bien, dérive au milieu, et le résultat est trop gros pour être relu	Un ticket à la fois
Laisser tourner 6 heures sans regarder	La dérive est cumulative et se découvre trop tard	Un point à chaque ticket terminé, même bref
Accepter « les tests passent » sans regarder les tests	Les tests peuvent avoir été écrits pour passer	Lire les tests des règles de gestion, pas le code
Changer d'avis en cours de ticket	Le contexte se brouille, le résultat mélange deux intentions	Finir le ticket, puis ouvrir un ticket de modification
Sessions interminables	La qualité baisse avec la longueur du contexte	Une session par ticket ou par petit groupe de tickets
Corriger soi-même en silence	L'erreur se reproduira	Faire corriger, et ajouter une ligne en section 9 du CLAUDE.md

5. Les cinq endroits où l'autonomie casse sur ce projet

Ils sont identifiables à l'avance. Prévoyez d'y être présent.

La synchronisation hors-ligne. Le point le plus difficile, et de loin. La résolution de conflits et la détection de doublons de machines créées hors réseau se testent mal, échouent silencieusement et se corrigent tard. Exigez les tests bout en bout `tests/e2e/offline/` avant l'implémentation, et validez vous-même le scénario en mode avion sur un vrai téléphone.

Le cloisonnement multi-société. Techniquement simple, catastrophique si mal fait. Faites écrire les tests d'isolation au lot 0, avant toute donnée métier, et faites-les tourner à chaque commit.

Les calendriers et les jours ouvrés. Chaque agent, laissé seul, finit par coder un calendrier lundi-vendredi. C'est faux pour Ducos. Le test doit exister avant la fonctionnalité.

Les imports Excel. Le contrôle avant application, les rejets annotés et l'annulation sous 24 heures sont faciles à implémenter à moitié. Testez avec un vrai fichier sale : lignes vides, doublons, colonnes déplacées, accents, dates au format français.

Le rendu PDF. Aucun test automatique ne dira qu'un rapport est laid ou qu'un tableau déborde. C'est le seul endroit du projet où le contrôle visuel humain est irremplaçable. Prévoyez-le à chaque évolution du modèle de document.

6. Backlog exécutable — lots 0 à 3

Chemin critique jusqu'à la mise en service terrain. Format : `[identifiant] but – critères d'acceptation`.

Lot 0 — Socle (3 semaines)

L0-01 — Initialiser le dépôt. Next.js 15 App Router, TypeScript strict, Tailwind, shadcn/ui, ESLint, Prettier, pnpm.

Acceptation : `pnpm build` et `pnpm typecheck` passent ; la page d'accueil affiche « CODIPLAN ».

L0-02 — Chaîne de vérification. Vitest, Playwright, script `pnpm verify` enchaînant typecheck, lint, test, test:isolation, build. Intégration continue déclenchée à chaque commit. *Acceptation* : `pnpm verify` passe ; un test volontairement faux fait échouer la commande et la CI.

L0-03 — Schéma multi-société. Tables `societe`, `devise`, `utilisateur`, `utilisateur_societe`. Chaque table métier future portera `societe_id`. *Acceptation* : migration appliquée ; `pnpm db:seed` crée deux sociétés, l'une en XPF, l'autre en EUR.

L0-04 — Politiques RLS. Sécurité au niveau des lignes sur toutes les tables portant `societe_id`, pilotée par une variable de session positionnée par la couche d'accès. *Acceptation* : une requête sans société positionnée retourne zéro ligne, et non l'ensemble des lignes.

L0-05 — Tests d'isolation. Répertoire `tests/isolation/`. Pour chaque ressource : lecture, écriture et suppression tentées depuis une autre société. *Acceptation* : `pnpm test:isolation` couvre au moins 10 scénarios et passe ; retirer le filtre société dans le code fait échouer les tests.

L0-06 — Authentification et rôles. Better Auth, sessions serveur, rôles par société via `utilisateur_societe`, MFA sur les rôles administrateur et direction. *Acceptation* : un utilisateur habilité sur A ne peut pas basculer sur B ; tout changement de société active est journalisé.

L0-07 — Module monétaire. `lib/money` : `formatMoney`, `addMoney`, arrondi selon les décimales de la devise. Point de passage unique. *Acceptation* : tests unitaires — 7 000 XPF s'affiche « 7 000 XPF » sans décimale, 100 EUR affiche « 100,00 € » ; toute conversion ligne à ligne lève une erreur.

L0-08 — Module calendrier. `lib/calendar` : calendriers par site, jours fériés paramétrables et travaillables, calcul des jours et heures ouvrés. *Acceptation* : tests — le samedi est ouvré pour Ducos et non pour Koné ; un férié marqué travaillé compte comme ouvré.

L0-09 — Thématisation par société. Couleurs, logo et mentions issus du paramétrage de la société active. *Acceptation* : basculer de société change l'identité visuelle sans redéploiement.

L0-10 — Journal d'audit. Table et intercepteur Prisma capturant création, modification et suppression avec valeurs avant/après. *Acceptation* : toute écriture sur une table sensible produit une ligne d'audit ; test automatisé.

Lot 1 — Référentiels, tarification, imports (4 semaines)

L1-01 Clients — CRUD, code Winpro comme clé de rapprochement, recherche. **L1-02** Sites — adresses, zones géographiques, consignes d'accès, calendriers d'ouverture, temps de trajet par agence. **L1-03** Contacts — rôles, préférences de notification. **L1-04** Techniciens — compétences, habilitations avec expiration, coût horaire, calendrier de travail. *Acceptation* : une habilitation expirée est signalée ; test sur RG-PLA-04. **L1-05** Familles et modèles de matériel — référentiel partageable (`societe_id` nullable), pièces d'usure, périodicités. **L1-06** Prestations et forfaits — catalogue conforme au §4.3, conditions d'application. *Acceptation* : un forfait dont les conditions ne sont pas remplies n'est pas proposé ; test sur RG-TAR-06. **L1-07** Taux horaire — valeur par société, surcharge par technicien, type et contrat, historisation. *Acceptation* : modifier le taux ne change pas les interventions déjà valorisées ; test sur RG-TAR-04. **L1-08** Moteur d'import — lecture Excel, mise en correspondance des colonnes, contrôle, rapport, application, annulation sous 24 h. *Acceptation* : un fichier de 300 lignes avec 5 erreurs produit un rapport exact ; l'annulation restaure l'état antérieur ; tests sur RG-IMP-01 à 05. **L1-09** Imports clients, sites, contacts, modèles, prestations — modèles Excel téléchargeables et documentés. **L1-10** Import de l'historique des ventes matériel — pré-crédation de fiches machines incomplètes. *Acceptation* : les fiches créées sont marquées `complet = false` et remontent dans une file de complétion.

Lot 2 — Parc et interventions (4 semaines)

L2-01 Fiche machine — création avec 3 champs obligatoires seulement (RG-PAR-02), unicité du couple modèle/n° de série par société. **L2-02** QR codes — génération, jeton, résolution, planche d'étiquettes imprimable. **L2-03** Compteurs — relevés, contrôle de non-régression (RG-PAR-04), projection d'atteinte de seuil. **L2-04** Documents machine — dépôt, visibilité client, marquage « embarqué mobile ». **L2-05** Historique machine — chronologie unifiée, conservée au changement de site (RG-PAR-03). **L2-06** Demandes — création, sources, horodatage de l'accusé de réception, qualification. **L2-07** Interventions — cycle de vie complet, transitions contrôlées. *Acceptation* : chaque transition interdite est refusée avec un message explicite ; tests sur RG-INT-01 à 11. **L2-08** Interventions multi-machines et multi-techniciens. **L2-09** Valorisation — forfait, temps passé, forfait plus heures excédentaires. **L2-10** File « en attente de pièce » — motif, référence attendue, date prévisionnelle, ancienneté.

Lot 3 — Planning et PWA (5 semaines)

L3-01 Vue calendrier ressources, glisser-déposer, redimensionnement. **L3-02** Contrôles à la pose — alertes non bloquantes, blocage strict sur habilitation expirée. **L3-03** File d'attente à planifier, tri par urgence et échéance, indicateur de risque. **L3-04** Absences, alerte de rupture de service à effectif unique, report groupé. **L3-05** Tournées — regroupement, ordonnancement, estimation des trajets. **L3-06** Socle PWA — manifeste, service worker, installabilité, écran hors-ligne. **L3-07** Cache local — IndexedDB, contenus du §13.2, synchronisation différentielle. **L3-08** File d'opérations et synchronisation — priorisation, reprise, indicateur d'état. *Acceptation : test bout en bout — intervention complète en mode avion puis synchronisation intégrale sans perte.* **L3-09** Résolution de conflits — application de I5, journalisation. *Acceptation : conflit simulé sur créneau et diagnostic, chacun résolu du bon côté.* **L3-10** Détection de doublons de machines créées hors ligne — signalement, jamais de suppression automatique. **L3-11** Scan QR et création de machine express — moins de 60 secondes, 3 champs. **L3-12** Recensement en série — enchaînement sans retour au menu, compteur de saisies. **L3-13** Saisie de rapport — checklist, temps, pièces, photos compressées, préconisations. **L3-14** Signature client — capture, horodatage, mention d'absence tracée. **L3-15** Génération et envoi du PDF — charte de la société émettrice, validation avant diffusion. *Acceptation : contrôle visuel humain obligatoire — aucun test automatique ne remplace ce point.*

Lots 4 à 7 — esquisse

Le même format s'applique. Contrats et générateur de propositions (lot 4), portail client et tableaux de bord (lot 5), exports et flux BI (lot 6), console éditeur et abonnements (lot 7). Découpez-les au moment de les aborder : un backlog écrit six mois à l'avance est périmé quand on y arrive.

7. Prompts d'amorçage

Première session

Lis `CLAUDE.md` et `docs/backlog.md`. Nous démarrons le lot 0. Commence par L0-01 et L0-02 uniquement — le dépôt et la chaîne de vérification. Ne code aucune fonctionnalité métier. Quand `pnpm verify` passe, arrête-toi et fais-moi un point.

Session de ticket

Ticket L2-07 — cycle de vie des interventions. Relis le §7/M3 du cahier des charges et les règles RG-INT-01 à RG-INT-11. Écris d'abord les tests, une par règle, en indiquant le numéro de règle en commentaire. Puis implémente. Termine par `pnpm verify`, puis résume les décisions que tu as prises sans me demander.

Session de reprise après une pause

Lis `CLAUDE.md`, `docs/decisions/` et les 10 derniers commits. Dis-moi où en est le projet et quel est le prochain ticket, avant de coder quoi que ce soit.

Après une erreur corrigée

Ajoute une ligne en section 9 du `CLAUDE.md` décrivant l'erreur et la règle à retenir, en une phrase.

8. Ce qui reste irréductiblement à vous

Aucun dispositif ne délègue ces quatre choses.

- **Les arbitrages métier.** Un montant de forfait, un délai d'engagement, une règle de facturation ne se déduisent pas : ils se décident.
 - **Le jugement sur l'ergonomie terrain.** Savoir si un technicien remplira réellement ce formulaire sous la pluie, avec des gants, demande de connaître le métier.
 - **La confrontation au réel.** Le premier vrai client, le premier vrai parc, le premier rapport contesté révéleront des choses qu'aucune spécification ne contenait.
 - **La décision d'arrêter.** Savoir qu'une fonctionnalité coûte plus qu'elle ne rapporte est un jugement, pas un calcul.
-

9. En une page

1. Construire la boucle de vérification **avant** le code métier — c'est elle qui rend l'autonomie possible.
2. Poser le `CLAUDE.md` à la racine et y inscrire les invariants **et les sujets sur lesquels s'arrêter**.
3. Découper en tickets dont chaque critère d'acceptation est vérifiable par une machine.
4. Sanctuariser les quatre gardiens : isolation, monnaie, calendrier, hors-ligne.
5. Un ticket à la fois, une session courte, un résumé des décisions implicites à chaque clôture.
6. Un point de contrôle humain par lot — utiliser le produit, pas seulement lire le rapport.
7. Chaque erreur corrigée devient une ligne dans le `CLAUDE.md`.
8. Être présent sur les cinq points de rupture : hors-ligne, cloisonnement, calendriers, imports, PDF.

Guide de pilotage CODIPLAN — CODIMA NC, Direction d'Exploitation — 18 août 2026